Surprise Quiz–2 : Solutions

1. **Which of the following statements(one or more) are correct regarding the repeated squaring technique and exponentiation are true based on the provided material?**

   A. The standard school method for computing a^n requires $n-1$ multiplications.

   B. Repeated squaring can reduce the number of multiplications for a^{16} to exactly 4.

   C. The recursive implementation $T(n) \leq T(n/2) + 2$ applies when n is even.

   D. The repeated squaring technique always yields the absolute minimum number of multiplications for any n.

   E. Repeated squaring is versatile enough to be applied to operations beyond simple integers, such as Matrix powering.

   Solution: A, B, C, E

   Explanation: While repeated squaring is highly efficient, there is no easy answer for the absolute minimum number of multiplications required for any n. For instance, a^{15} can be computed in 5 multiplications, whereas standard repeated squaring might suggest more.

2. **You are given an array `A` of length `n` (0-indexed).**

   `A[i]` tells you the maximum jump length from index `i`.

   You start at index `0` and want to reach index $n-1$ using the minimum number of jumps. Your friend has proposed a greedy solutions and a proof of correctness for their greedy solution. Choose the options that are correct regarding the solution.

   Proposed Solution: Greedily Choose Indices

   From the current index i, look at all the reachable indices $j \in [i+1, i+A[i]]$ and jump to the index j, that maximises $j + A[j]$.

```
jumps = 0, i = 0
while i < n-1:
    choose j in [i+1, i+A[i]] maximizing (j + A[j])
    i = j; jumps++
return jumps
```

   Proof of Correctness: By maximising the reach in two jumps, we ensure we cover the maximum possible distance at every even-numbered jump. We are minimizing the distance to the last index as aggressively as possible. Repeating this choice at every step guarantees the minimum number of jumps.
   Choose the correct option(one or more).

   A. Solution is correct

   B. Solution is incorrect

   C. The solution is incorrect because making the longest even numbered jump does not guarantee the optimal behaviour since we are not considering all the cases

   D. The solution is incorrect because it assumes that maximizing two jump reach at each step leads to global optimum, which does not necessarily hold.

   Solution: A

   Explanation: Neetcode video on Jump Game II Leetcode Problem 45

   Problem Summary

   Given an array A of length n, where $A[i]$ denotes the maximum jump length from index i, we must reach index $n-1$ using the minimum number of jumps.

   Proposed greedy strategy:

   > From the current index i, choose
   >
   > $$j \in [i+1,\ i+A[i]]$$
   >
   > that maximizes $(j + A[j])$.

   This means we always choose the next position that gives the **maximum two–step reach**.

   Proof 1: BFS / Layer Interpretation

   Interpret each index as a node of a graph.

- Edge from $i \to j$ if $j \leq i + A[i]$
- Each jump = crossing one edge

Thus the task becomes finding the **shortest path in an unweighted graph**, which is solved optimally using BFS.

All nodes reachable in one jump form a layer (interval). Choosing the index with maximum $j + A[j]$ expands the next layer as far as possible.

Hence this greedy exactly simulates BFS traversal.

Therefore it gives the minimum number of jumps.

Proof 2: Greedy Stays Ahead (Exchange Argument)

Let

- G = greedy solution
- O = optimal solution

At a step, suppose optimal picks index k while greedy picks j.

By greedy choice:

$$j + A[j] \geq k + A[k]$$

So greedy can reach at least as far as optimal in the next step.

Hence:

$$\text{reach}_G \geq \text{reach}_O \quad \text{at every step}$$

Greedy never falls behind optimal. Any position where the optimal solution can jump to, greedy can jump there as well. Therefore greedy uses no more jumps than optimal.

Greedy is optimal.

3. **Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_{100}\}$ and $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$. The weight of the edge (v_i, v_j) is $|i - j|$.**

   The weight of the minimum spanning tree of G is:

   A. 99

   B. 100

   C. 98

   D. 101

   Solution: A

   Explanation: The edge weight is $|i-j|$. The minimum possible weight is 1, which occurs between consecutive vertices:

$$(v_1, v_2), (v_2, v_3), \ldots, (v_{99}, v_{100}).$$

   A minimum spanning tree on 100 vertices requires $100 - 1 = 99$ edges. Choosing these edges connects all vertices without forming cycles, and every edge has weight 1. Hence, the total weight is:

$$99 \times 1 = 99.$$

4. **Which DP transition correctly defines the 0/1 Knapsack problem?**

   A. $OPT[i][w] = \max\big(OPT[i-1][w],\ OPT[i][w-w_i] + v_i\big)$

   B. $OPT[i][w] = \max\big(OPT[i-1][w] + v_i,\ OPT[i][w-w_i]\big)$

   C. $OPT[i][w] = \max\big(OPT[i-1][w],\ OPT[i-1][w-w_i]\big) + v_i$

   D. $OPT[i][w] = \max\big(OPT[i-1][w],\ OPT[i-1][w-w_i] + v_i\big)$

   Solution: D

   Explanation:

   In the 0/1 Knapsack problem, $OPT[i][w]$ denote the maximum value achievable using the first i items with capacity w. The recurrence considers two choices:

   (a) Do not include item i: value is $OPT[i-1][w]$

   (b) Include item i (if $w_i \leq w$): value is $OPT[i-1][w-w_i] + v_i$

   Thus, the correct DP transition is $OPT[i][w] = \max\big(OPT[i-1][w],\ OPT[i-1][w-w_i] + v_i\big)$.

5. **You are given $n = 6$ time intervals $I_1, \ldots, I_6$, already sorted by increasing start time. Each interval is $I_j = (s_j, e_j)$, where s_j is the start time and e_j is the end time.**

Define

$$p(j) = \min\{k > j \ : \ s_k > e_j\},$$

and if no such k exists, set $p(j) = n + 1$.

Given the intervals:

Interval	(s_j, e_j)	index
I_1	$(0, 2)$	1
I_2	$(1, 5)$	2
I_3	$(3, 4)$	3
I_4	$(5, 9)$	4
I_5	$(6, 7)$	5
I_6	$(8, 10)$	6

Fill in the blanks:

$$p(1) = \_\_\_\_\_, \quad p(2) = \_\_\_\_\_, \quad p(3) = \_\_\_\_\_.$$

Solution: p(1)=3, p(2)=5, p(3)=4

Explanation:

We are given intervals $I_1, \ldots, I_6$ sorted by increasing start time. For each j, define

$$p(j) = \min\{k > j \ : \ s_k > e_j\},$$

and if no such k exists, set $p(j) = n + 1$.

Compute $p(1)$. $I_1 = (0, 2)$, so $e_1 = 2$. We need the first $k > 1$ such that $s_k > 2$. Checking in order:

- $I_2 = (1, 5)$ has $s_2 = 1$, and $1 > 2$ is false.
- $I_3 = (3, 4)$ has $s_3 = 3$, and $3 > 2$ is true.

Hence, $p(1) = 3$.

Compute $p(2)$. $I_2 = (1, 5)$, so $e_2 = 5$. We need the first $k > 2$ such that $s_k > 5$. Checking in order:

- $I_3 = (3, 4)$ has $s_3 = 3$, and $3 > 5$ is false.
- $I_4 = (5, 9)$ has $s_4 = 5$, and $5 > 5$ is false (strict inequality).
- $I_5 = (6, 7)$ has $s_5 = 6$, and $6 > 5$ is true.

Hence, $p(2) = 5$.

Compute $p(3)$. $I_3 = (3, 4)$, so $e_3 = 4$. We need the first $k > 3$ such that $s_k > 4$. Checking in order:

- $I_4 = (5, 9)$ has $s_4 = 5$, and $5 > 4$ is true.

Hence, $p(3) = 4$.

$$\boxed{p(1) = 3, \quad p(2) = 5, \quad p(3) = 4}$$

6. **Let G be a connected undirected weighted graph. Consider the following statements:**

   - **S1**: Suppose that the minimum edge weight in G is not necessarily unique. Then there exists a minimum-weight edge in G that appears in every minimum spanning tree of G.
   - **S2**: If every edge in G has distinct weight, then G has a unique minimum spanning tree.

   Which one of the following options is correct?

   A. Both S1 and S2 are true

   B. S1 is true and S2 is false

   C. S1 is false and S2 is true

   D. Both S1 and S2 are false

   Solution: C

   Explanation:

   S1: False.

   Suppose the minimum edge weight is not unique; that is, multiple edges share the same smallest weight. In such a situation, it is possible to construct different minimum spanning trees by choosing different edges among these equal-weight edges.

   This follows from the cut property: an edge is guaranteed to be present in every MST only if it is the *unique* lightest edge crossing some cut of the graph. When several edges have the same minimum weight, none of them is forced to be chosen, since replacing one with another does not change the total weight of the spanning tree.

   Hence, no particular minimum-weight edge is guaranteed to appear in every MST.

   S2: True.

   If all edge weights are distinct, then for every cut in the graph there is exactly one lightest edge. By the cut property, that edge must belong to the MST. Because every greedy choice is uniquely determined, no alternative spanning tree can have the same minimum weight.

   Therefore, the minimum spanning tree is unique.

7. **Let $T(d)$ be the time required to evaluate a degree $d-1$ polynomial at all d-th roots of unity using the Fast Fourier Transform (FFT). Which of the following recurrences correctly describes $T(d)$?**

   A. $T(d) = T(d-1) + O(d)$

   B. $T(d) = 2T(d/2) + O(d)$

   C. $T(d) = T(d/2) + O(1)$

   D. $T(d) = 2T(d/2) + O(1)$

   Solution: B

   Explanation:

   When evaluating a degree $d-1$ polynomial at all d-th roots of unity using the Fast Fourier Transform (FFT), the algorithm splits the polynomial into its even and odd parts, resulting in two subproblems of size $d/2$. The cost of combining the results is $O(d)$. Therefore, the recurrence relation is $T(d) = 2T(d/2) + O(d)$.

8. **When developing a dynamic programming algorithm, the sequence of steps followed is:**

   A. Construct an optimal solution from computed information.

   B. Recursively define the value of an optimal solution.

   C. Characterize the structure of an optimal solution.

   D. Compute the value of an optimal solution, typically in a bottom-up fashion.

   Write the steps in the correct order.

   Solution: $\boxed{C \rightarrow B \rightarrow D \rightarrow A}$

   Explanation:

   In dynamic programming, we typically proceed as follows:

   (C) First, *characterize the structure of an optimal solution* (identify optimal substructure).

   (B) Next, *recursively define the value of an optimal solution* using a recurrence over smaller subproblems.

   (D) Then, *compute the value of an optimal solution*, usually in a bottom-up manner (or via memoization).

   (A) Finally, *construct an optimal solution* by backtracking from the computed information.

9. **Your friend tried to solve the problem of Maximum Contiguous Subarray Sum using Dynamic Programming, but is unable to complete the recurrence. Choose the correct recurrence to solve the problem for them. The input in an integer array A of length n, it has 0-indexing. The elements of the array can be positive, negative or zero.**
Fill in the blank in the pseudocode.
Note: `dp[i]` stores the maximum subarray sum for the subarray `A[0,...,i]` ending at the index `i`.

```
function MaxSubarraySum(A):
    n = length(A)
    dp = array of size n

    dp[0] = A[0]
    ans = dp[0]

    for i = 1 to n-1:
        dp[i] = ________________________  // Fill in the blank
        ans = max(ans, dp[i])

    return ans
```

Solution:

$$\boxed{dp[i] = \max(A[i], dp[i-1] + A[i])}$$

Explanation:

Definition

Let

$$dp[i] = \text{maximum subarray sum ending at index } i$$

Recurrence

At each index there are 3 options to choose from:

- start a new subarray at i
- extend the previous subarray

Hence,

$$\boxed{dp[i] = \max(A[i],\ dp[i-1] + A[i])}$$

Fill in the Blank

```
dp[i] = max(A[i], dp[i-1] + A[i])
```

Note

This is Kadane's Algorithm and runs in

$$O(n) \text{ time and } O(n) \text{ space}$$

(or $O(1)$ space with optimization).

10. **Which statements(one or more) accurately describe the complexity and methodology of Karatsuba and Toom-Cook multiplication?**

    A. Karatsuba's algorithm disproved Kolmogorov's conjecture that $O(n^2)$ was the necessary time for integer multiplication.

    B. The recurrence relation for Karatsuba's method is $T(n) = 3T(n/2) + O(n)$, resulting in $O(n^{1.585})$.

    C. In Toom-Cook multiplication, breaking integers into three parts ($n/3$ bits) and using five multiplications results in a complexity of $O(n^{1.46})$.

    D. The Toom-Cook method can achieve better time complexity by increasing the number of parts k, potentially reaching $O(n^{1+\epsilon})$ for any constant $\epsilon > 0$.

    Solution: A,B,C,D

    Explanation:

    All statements accurately reflect the progression of multiplication algorithms from Karatsuba's initial breakthrough to the theoretical $O(n \log n)$ bound achieved in 2019.